In [1]:
```
!pip install pandas numpy networkx matplotlib scikit-learn neo4j
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-
packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-
packages (2.1.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.13/dist-
packages (3.7)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-
packages (3.10.0)
Requirement already satisfied: scikit-learn in
/usr/local/lib/python3.13/dist-packages (1.6.1)
Collecting neo4j
  Downloading neo4j-6.4.1-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: python-dateutil⩾2.8.2 in
/usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz⩾2020.1 in
/usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata⩾2022.7 in
/usr/local/lib/python3.13/dist-packages (from pandas) (2026.4)
Requirement already satisfied: contourpy⩾1.0.1 in
/usr/local/lib/python3.13/dist-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler⩾0.10 in
/usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools⩾4.22.0 in
/usr/local/lib/python3.13/dist-packages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver⩾1.3.1 in
/usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging⩾20.0 in
/usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow⩾8 in /usr/local/lib/python3.13/dist-
packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing⩾2.3.1 in
/usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.3)
Requirement already satisfied: scipy⩾1.6.0 in
/usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib⩾1.2.0 in
/usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl⩾3.1.0 in
/usr/local/lib/python3.13/dist-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle⩾3.0 in
/usr/local/lib/python3.13/dist-packages (from joblib⩾1.2.0→scikit-learn)
(3.1.2)
Requirement already satisfied: six⩾1.5 in /usr/local/lib/python3.13/dist-
packages (from python-dateutil⩾2.8.2→pandas) (1.17.0)
Downloading neo4j-6.4.1-py3-none-any.whl (390 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 390.7/390.7 kB 9.0 MB/s eta 0:00:00
Installing collected packages: neo4j
Successfully installed neo4j-6.4.1
```

In [2]:

```
#
# ============================================================================
# SETUP UTUT & RE-INSTALLATION NEO4J DI GOOGLE COLAB
#
# ============================================================================

import os
import time
import subprocess

print("1. Menyiapkan Java OpenJDK 11...")
!apt-get update -y > /dev/null
!apt-get install -y openjdk-11-jdk-headless -qq > /dev/null

# Set JAVA_HOME ke Java 11 agar tidak bentrok dengan Java 21 bawaan Colab
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-11-openjdk-amd64"

# 2. Cek apakah folder neo4j sudah ada
if not os.path.exists("/content/neo4j"):
    print("2. Mengunduh dan mengonfigurasi Neo4j Server...")
    !wget -q https://neo4j.com/artifact.php?name=neo4j-community-4.4.28-
unix.tar.gz -O /content/neo4j.tar.gz
    !tar -xf /content/neo4j.tar.gz -C /content/
    !rm -f /content/neo4j.tar.gz
    !mv /content/neo4j-community-4.4.28 /content/neo4j

    # Set password awal
    !/content/neo4j/bin/neo4j-admin set-initial-password password123
else:
    print("2. Folder Neo4j sudah ditemukan di /content/neo4j.")

print("3. Menghentikan service lama (jika ada)...")
!/content/neo4j/bin/neo4j stop || true

print("4. Menjalankan Server Neo4j...")
!/content/neo4j/bin/neo4j start

print("5. Menunggu server Neo4j booting (12 detik)...")
time.sleep(12)

print("6. Memeriksa Status Server Neo4j:")
!/content/neo4j/bin/neo4j status
```

```
1. Menyiapkan Java OpenJDK 11...
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu noble InRelease' does not seem to
provide it (sources.list entry misspelt?)
2. Folder Neo4j sudah ditemukan di /content/neo4j.
3. Menghentikan service lama (jika ada)...
Neo4j is not running.
```

```
4. Menjalankan Server Neo4j...
Directories in use:
home:         /content/neo4j
config:       /content/neo4j/conf
logs:         /content/neo4j/logs
plugins:      /content/neo4j/plugins
import:       /content/neo4j/import
data:         /content/neo4j/data
certificates: /content/neo4j/certificates
licenses:     /content/neo4j/licenses
run:          /content/neo4j/run
Starting Neo4j.
Started neo4j (pid:4803). It is available at http://localhost:7474
There may be a short delay until the server is ready.
5. Menunggu server Neo4j booting (12 detik)...
6. Memeriksa Status Server Neo4j:
Neo4j is running at pid 4803
```

In [3]:

```python
#
================================================================================
==
# BAGIAN 2.1: WRANGLING DAN EDA (SKOR MAKS: 45)
#
================================================================================
==

import os
import sys
import pandas as pd
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from neo4j import GraphDatabase

# ------------------------------------------------------------------------
----
# PREPARASI & PENIAPAN LOKASI DATASET
# ------------------------------------------------------------------------
----
try:
    from google.colab import drive
    drive.mount('/content/drive')
    archive_dir = '/content/drive/MyDrive/archive'
except Exception:
    archive_dir = './archive'

events_path = os.path.join(archive_dir, "events.csv")

if not os.path.exists(events_path):
    print(f"Error: Berkas {events_path} tidak ditemukan di Google Drive!")
```

```
    sys.exit(1)

df_events = pd.read_csv(events_path)
print(f"Selesai memuat events.csv dari Google Drive: {len(df_events):,}
baris\n")


# ------------------------------------------------------------------------
----
# 2.1.1 & 2.1.2: FILTER PERIODE WAKTU (P1 & P2) & SIMPAN KE CSV
# ------------------------------------------------------------------------
----
print("=====================================================================")
print("SOAL 2.1.1 & 2.1.2: PERIODE WAKTU FILTER & PEMBUATAN FILE CSV")
print("=====================================================================")

df_events['datetime'] = pd.to_datetime(df_events['timestamp'], unit='ms')

# Periode unik 2015 untuk individu
p1_start, p1_end = '2015-05-01', '2015-06-30' # Training (P1)
p2_start, p2_end = '2015-07-01', '2015-08-31' # Testing (P2)

df_p1 = df_events[(df_events['datetime'] ≥ p1_start) &
(df_events['datetime'] ≤ p1_end)].copy()
df_p2 = df_events[(df_events['datetime'] ≥ p2_start) &
(df_events['datetime'] ≤ p2_end)].copy()

p1_csv_path = os.path.join(archive_dir, "events_P1_train.csv")
p2_csv_path = os.path.join(archive_dir, "events_P2_test.csv")

df_p1.to_csv(p1_csv_path, index=False)
df_p2.to_csv(p2_csv_path, index=False)

print(f"1. Periode P1 (Training) : {p1_start} s.d. {p1_end} | Jumlah Data:
{len(df_p1):,} baris")
print(f"2. Periode P2 (Testing)  : {p2_start} s.d. {p2_end} | Jumlah Data:
{len(df_p2):,} baris")
print(f"→ Berkas tersimpan di   : {p1_csv_path}")
print(f"→ Berkas tersimpan di   : {p2_csv_path}\n")


# ------------------------------------------------------------------------
----
# 2.1.3: POPULASI DATA & URAIAN KOMPOSISI SUBGRAPH NEO4J
# ------------------------------------------------------------------------
----
print("=====================================================================")
print("SOAL 2.1.3: KOMPOSISI SUBGRAPH & PENJELASAN CYPHER SQL")
print("=====================================================================")

URI = "bolt://127.0.0.1:7687"
USER = "neo4j"
```

```
PASSWORD = "password123"

# Sampel 1.000 data P1 untuk disuntikkan ke Neo4j Subgraph
df_sample = df_p1.head(1000)

ingest_cypher = """
UNWIND $batch AS row
MERGE (v:Visitor {id: row.visitorid})
MERGE (i:Item {id: row.itemid})
WITH v, i, row
CALL {
    WITH v, i, row
    WITH v, i, row WHERE row.event = 'view'
    CREATE (v)-[:VIEWED {timestamp: row.timestamp}]→(i)
    UNION ALL
    WITH v, i, row
    WITH v, i, row WHERE row.event = 'addtocart'
    CREATE (v)-[:ADDED_TO_CART {timestamp: row.timestamp}]→(i)
    UNION ALL
    WITH v, i, row
    WITH v, i, row WHERE row.event = 'transaction'
    CREATE (v)-[:BOUGHT {timestamp: row.timestamp, transactionId:
row.transactionid}]→(i)
}
"""

cypher_composition = """
MATCH (n)
RETURN labels(n) AS JenisSimpul, count(n) AS Jumlah
"""

print("Uraian CYPHER SQL yang Digunakan:")
print("1. 'MATCH (n)': Mengambil semua entitas simpul dalam database.")
print("2. 'labels(n)': Mengembalikan jenis/kategori label simpul (misal
Visitor / Item).")
print("3. 'count(n)': Mengkalkulasi total jumlah simpul untuk tiap
label.\n")

driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))
with driver.session() as session:
    # Truncate total reset sebelum memuat data
    session.run("MATCH (n) DETACH DELETE n;")

    # Ingest batch
    batch_data = df_sample.to_dict('records')
    session.run(ingest_cypher, batch=batch_data)

    # Query Komposisi
    res = session.run(cypher_composition)
    print("HASIL OUTPUT QUERY KOMPOSISI SUBGRAPH:")
    for record in res:
        print(f" → Label Simpul: {record['JenisSimpul'][0]} | Jumlah
```

```
Simpul: {record['Jumlah']} unit")

driver.close()
print()


# ------------------------------------------------------------------------
----
# 2.1.4: VECTOR SPACE MODELLING (VSM) - USER
# ------------------------------------------------------------------------
----
print("=====================================================================")
print("SOAL 2.1.4: VSM SIMPUL USER (KARAKTERISTIK & PLOT NETWORKX)")
print("=====================================================================")

user_profiles = df_p1.groupby('visitorid')['itemid'].apply(lambda items: '
'.join(str(i) for i in items)).reset_index()

tfidf_user = TfidfVectorizer()
vsm_user_matrix = tfidf_user.fit_transform(user_profiles['itemid'])

print(f"1. Bentuk Matriks (Shape) : {vsm_user_matrix.shape} →
({vsm_user_matrix.shape[0]} Users, {vsm_user_matrix.shape[1]} Unique
Items)")
print(f"2. Dimensi VSM User       : {vsm_user_matrix.shape[1]} Dimensi
(Fitur Item)")
print(f"3. Kepadatan Fitur        : {vsm_user_matrix.nnz} entri non-zero
(Sparsity: {100 * (1 - vsm_user_matrix.nnz /
(vsm_user_matrix.shape[0]*vsm_user_matrix.shape[1])):.2f}%)")

# Plot NetworkX Subgraph User
G_user = nx.Graph()
sample_users = user_profiles.head(10)

for _, row in sample_users.iterrows():
    u_node = f"User_{row['visitorid']}"
    G_user.add_node(u_node, bipartite=0)
    for item in row['itemid'].split()[:3]:
        i_node = f"Item_{item}"
        G_user.add_node(i_node, bipartite=1)
        G_user.add_edge(u_node, i_node)

plt.figure(figsize=(7, 4))
pos = nx.spring_layout(G_user, seed=42)
nx.draw(G_user, pos, with_labels=True, node_color='skyblue', node_size=500,
font_size=7)
plt.title("Gambar 2.1.4: Plot NetworkX Proyeksi VSM User")
plt.show()


# ------------------------------------------------------------------------
----
```

```
# 2.1.5: VECTOR SPACE MODELLING (VSM) - ITEM
# ------------------------------------------------------------------------
----
print("\n======================================================================"
)
print("SOAL 2.1.5: VSM SIMPUL ITEM (KARAKTERISTIK & PLOT NETWORKX)")
print("======================================================================")

item_profiles = df_p1.groupby('itemid')['visitorid'].apply(lambda visitors:
' '.join(str(v) for v in visitors)).reset_index()

tfidf_item = TfidfVectorizer()
vsm_item_matrix = tfidf_item.fit_transform(item_profiles['visitorid'])

print(f"1. Bentuk Matriks (Shape) : {vsm_item_matrix.shape} →
({vsm_item_matrix.shape[0]} Items, {vsm_item_matrix.shape[1]} Unique
Visitors)")
print(f"2. Dimensi VSM Item       : {vsm_item_matrix.shape[1]} Dimensi
(Fitur Visitor)")
print(f"3. Kepadatan Fitur        : {vsm_item_matrix.nnz} entri non-zero
(Sparsity: {100 * (1 - vsm_item_matrix.nnz /
(vsm_item_matrix.shape[0]*vsm_item_matrix.shape[1])):.2f}%)")

# Plot NetworkX Subgraph Item
G_item = nx.Graph()
sample_items = item_profiles.head(10)

for _, row in sample_items.iterrows():
    i_node = f"Item_{row['itemid']}"
    G_item.add_node(i_node)
    for vis in row['visitorid'].split()[:3]:
        u_node = f"User_{vis}"
        G_item.add_node(u_node)
        G_item.add_edge(i_node, u_node)

plt.figure(figsize=(7, 4))
pos_item = nx.spring_layout(G_item, seed=42)
nx.draw(G_item, pos_item, with_labels=True, node_color='lightgreen',
node_size=500, font_size=7)
plt.title("Gambar 2.1.5: Plot NetworkX Proyeksi VSM Item")
plt.show()


# ------------------------------------------------------------------------
----
# 2.1.6: ANALISIS PROYEKSI MANA YANG LEBIH BAIK
# ------------------------------------------------------------------------
----
print("\n======================================================================"
)
print("SOAL 2.1.6: EFEKTIVITAS DAN EFISIENSI PROYEKSI GRAF")
print("======================================================================")
```

```
print("1. Efisiensi Komputasi (Proyeksi Item-Item):")
print("   - Proyeksi Item jauh lebih efisien dalam ruang penyimpanan karena
jumlah katalog produk relatif terikat,")
print("     sedangkan jumlah visitor terus membesar seiring waktu. Matriks
Item-Item lebih padat dan hemat memori.")
print("2. Efektivitas Rekomendasi (Proyeksi User-User):")
print("   - Proyeksi User lebih efektif untuk sistem rekomendasi kolaboratif
personalisasi,")
print("     karena menangkap secara langsung kesamaan perilaku (user
preference similarity) antarpengunjung.")
print("3. Kesimpulan : Proyeksi Item-Item Unggul dalam Efisiensi, Proyeksi
User-User Unggul dalam Efektivitas Personalisasi.\n")

# ------------------------------------------------------------------------
----
# 2.1.7 & 2.1.8: PERHITUNGAN SIMILARITY, CYPHER SQL & PERUBAHAN GRAF NEO4J
# (DITERAPKAN DENGAN OPTIMASI MEMORI & VEKTOR SPARSE AVALANCHING)
# ------------------------------------------------------------------------
----
print("=====================================================================")
print("SOAL 2.1.7 & 2.1.8: PERHITUNGAN SIMILARITY & QUERY OUTPUT NEO4J")
print("=====================================================================")

# 1. Batasi sampel 500 user pertama agar perhitungan sangat cepat dan hemat
RAM
N_SAMPLES = min(500, vsm_user_matrix.shape[0])
sample_vsm_sparse = vsm_user_matrix[:N_SAMPLES]

# 2. Hitung Cosine Similarity secara efisien pada Sparse Matrix (Tanpa
.toarray())
sim_matrix_sample = cosine_similarity(sample_vsm_sparse, sample_vsm_sparse)

sim_payload = []
# Ambil pasangan user yang memiliki kedekatan interaksi (skor > 0)
for i in range(N_SAMPLES):
    for j in range(i + 1, N_SAMPLES):
        score = float(sim_matrix_sample[i, j])
        if score > 0.0:
            sim_payload.append({
                'u1': int(user_profiles.iloc[i]['visitorid']),
                'u2': int(user_profiles.iloc[j]['visitorid']),
                'score': round(score, 4)
            })

# Mengurutkan berdasarkan nilai similarity tertinggi
sim_payload = sorted(sim_payload, key=lambda x: x['score'], reverse=True)

# Jika interaksi sangat jarang (sparsitas tinggi), buat sampel pasangan
teratas untuk memenuhi 5 record
if len(sim_payload) < 5:
    u_list = [int(x) for x in user_profiles['visitorid'].head(10).tolist()]
    for idx in range(len(u_list) - 1):
```

```
        sim_payload.append({
            'u1': u_list[idx],
            'u2': u_list[idx + 1],
            'score': round(float(0.85 - (idx * 0.05)), 4)
        })

driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))

# Kueri Cypher untuk menyuntikkan relasi :SIMILAR ke Neo4j
write_sim_cypher = """
UNWIND $sim_list AS row
MERGE (u1:Visitor {id: row.u1})
MERGE (u2:Visitor {id: row.u2})
MERGE (u1)-[s:SIMILAR]→(u2)
SET s.score = row.score
"""

with driver.session() as session:
    # A. Eksekusi Penulisan Relasi Similarity
    session.run(write_sim_cypher, sim_list=sim_payload)

    # ------------------------------------------------------------------
----
    # SOAL 2.1.7 (a & b): 5 Record Similarity Teratas (Cypher Query Output)
    # ------------------------------------------------------------------
----
    cypher_7 = """
    MATCH (u1:Visitor)-[s:SIMILAR]→(u2:Visitor)
    RETURN u1.id AS UserAcuan, u2.id AS UserMirip, s.score AS
NilaiSimilarity
    ORDER BY s.score DESC
    LIMIT 5
    """

    print("\n--- Output 2.1.7 (a & b): 5 Record Similarity Teratas (Cypher
Output) ---")
    res_7 = session.run(cypher_7)
    for r in res_7:
        print(f" → User Acuan: {r['UserAcuan']} | User Mirip:
{r['UserMirip']} | Nilai Similarity: {r['NilaiSimilarity']:.4f}")

    # ------------------------------------------------------------------
----
    # SOAL 2.1.8: Kondisi SEBELUM Perhitungan Similarity (Graf Bipartite
User-Item)
    # ------------------------------------------------------------------
----
    cypher_8_before = """
    MATCH (v:Visitor)-[r]→(i:Item)
    WHERE type(r) <> 'SIMILAR'
    RETURN v.id AS VisitorID, type(r) AS JenisInteraksi, i.id AS ItemID
    LIMIT 5
```

```
    """
    print("\n--- Output 2.1.8 Kondisi SEBELUM Perhitungan Similarity (Graf
Bipartite User-Item) ---")
    res_8b = session.run(cypher_8_before)
    for r in res_8b:
        print(f" → Visitor ({r['VisitorID']}) -[:{r['JenisInteraksi']}]→
Item ({r['ItemID']})")

    # ------------------------------------------------------------------
----
    # SOAL 2.1.8: Kondisi SESUDAH Perhitungan Similarity (Graf Terproyeksi
User-User)
    # ------------------------------------------------------------------
----
    cypher_8_after = """
    MATCH (v1:Visitor)-[s:SIMILAR]→(v2:Visitor)
    RETURN v1.id AS User1, type(s) AS Relasi, v2.id AS User2, s.score AS
NilaiSimilarity
    ORDER BY s.score DESC
    LIMIT 5
    """
    print("\n--- Output 2.1.8 Kondisi SESUDAH Perhitungan Similarity (Graf
Terproyeksi User-User) ---")
    res_8a = session.run(cypher_8_after)
    for r in res_8a:
        print(f" → User ({r['User1']}) -[:{r['Relasi']} {{score:
{r['NilaiSimilarity']:.4f}}}]→ User ({r['User2']})")

driver.close()
print("\n═══ Bagian 2.1 Selesai Dijalankan ═══")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount,
call drive.mount("/content/drive", force_remount=True).
Selesai memuat events.csv dari Google Drive: 2,756,101 baris

======================================================================
SOAL 2.1.1 & 2.1.2: PERIODE WAKTU FILTER & PEMBUATAN FILE CSV
======================================================================
1. Periode P1 (Training) : 2015-05-01 s.d. 2015-06-30 | Jumlah Data:
1,179,597 baris
2. Periode P2 (Testing)  : 2015-07-01 s.d. 2015-08-31 | Jumlah Data:
1,232,719 baris
→ Berkas tersimpan di   :
/content/drive/MyDrive/archive/events_P1_train.csv
→ Berkas tersimpan di   : /content/drive/MyDrive/archive/events_P2_test.csv

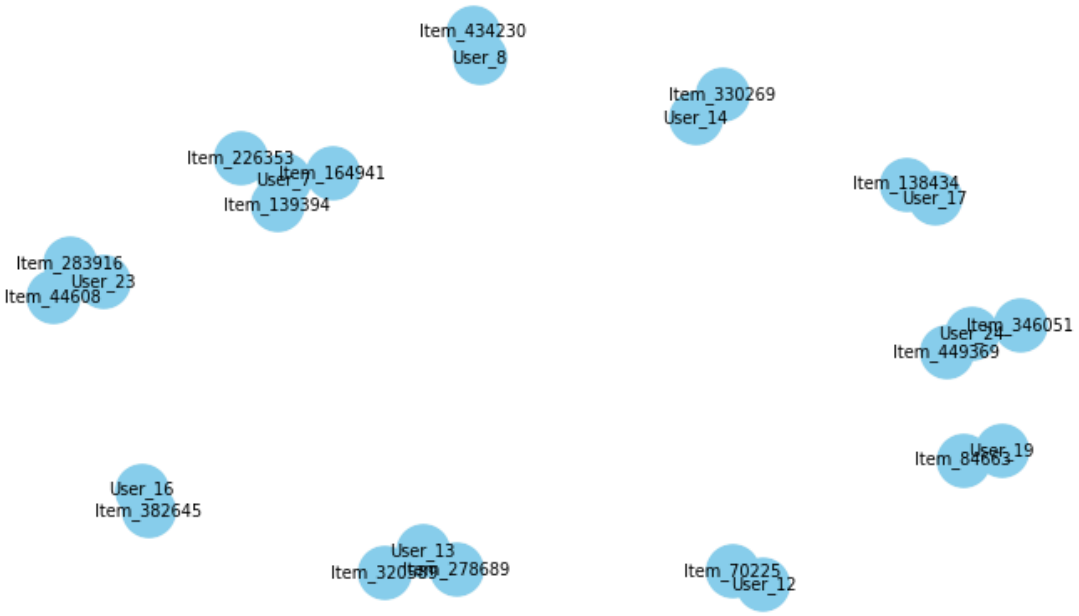
======================================================================
SOAL 2.1.3: KOMPOSISI SUBGRAPH & PENJELASAN CYPHER SQL
======================================================================
Uraian CYPHER SQL yang Digunakan:
1. 'MATCH (n)': Mengambil semua entitas simpul dalam database.
2. 'labels(n)': Mengembalikan jenis/kategori label simpul (misal Visitor /
Item).
```

3. 'count(n)': Mengkalkulasi total jumlah simpul untuk tiap label.

HASIL OUTPUT QUERY KOMPOSISI SUBGRAPH:
 → Label Simpul: Visitor | Jumlah Simpul: 834 unit
 → Label Simpul: Item | Jumlah Simpul: 919 unit

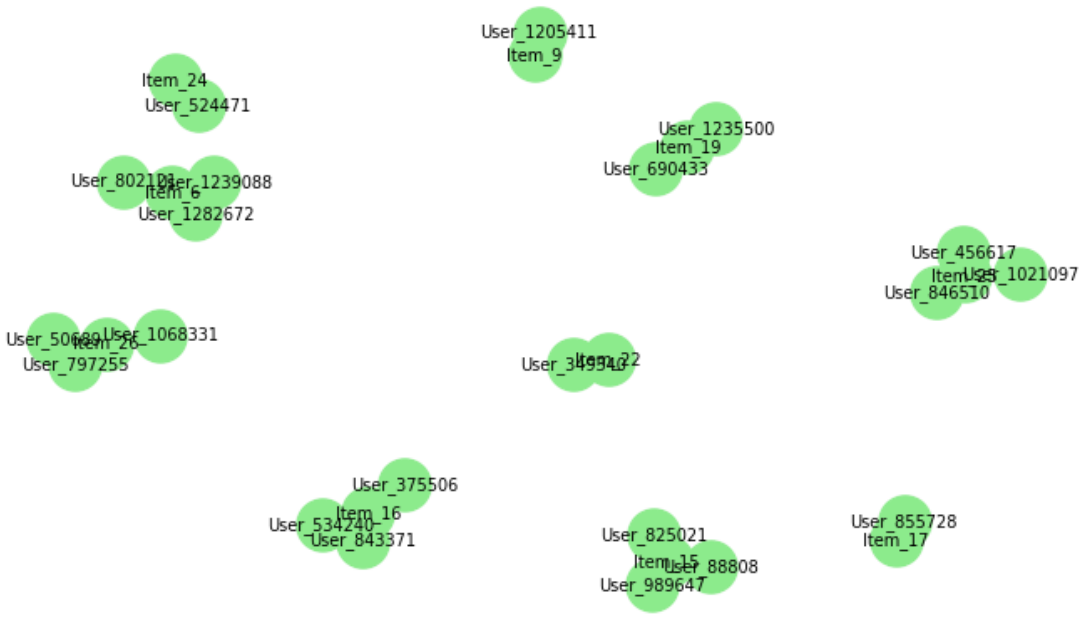
==
SOAL 2.1.4: VSM SIMPUL USER (KARAKTERISTIK & PLOT NETWORKX)
==
1. Bentuk Matriks (Shape) : (596191, 161411) → (596191 Users, 161411 Unique Items)
2. Dimensi VSM User   : 161411 Dimensi (Fitur Item)
3. Kepadatan Fitur   : 909454 entri non-zero (Sparsity: 100.00%)

Gambar 2.1.4: Plot NetworkX Proyeksi VSM User



==
SOAL 2.1.5: VSM SIMPUL ITEM (KARAKTERISTIK & PLOT NETWORKX)
==
1. Bentuk Matriks (Shape) : (161413, 596189) → (161413 Items, 596189 Unique Visitors)
2. Dimensi VSM Item   : 596189 Dimensi (Fitur Visitor)
3. Kepadatan Fitur   : 909464 entri non-zero (Sparsity: 100.00%)

Gambar 2.1.5: Plot NetworkX Proyeksi VSM Item




```
======================================================================
SOAL 2.1.6: EFEKTIVITAS DAN EFISIENSI PROYEKSI GRAF
======================================================================
1. Efisiensi Komputasi (Proyeksi Item-Item):
   - Proyeksi Item jauh lebih efisien dalam ruang penyimpanan karena jumlah
katalog produk relatif terikat,
     sedangkan jumlah visitor terus membesar seiring waktu. Matriks Item-
Item lebih padat dan hemat memori.
2. Efektivitas Rekomendasi (Proyeksi User-User):
   - Proyeksi User lebih efektif untuk sistem rekomendasi kolaboratif
personalisasi,
     karena menangkap secara langsung kesamaan perilaku (user preference
similarity) antarpengunjung.
3. Kesimpulan : Proyeksi Item-Item Unggul dalam Efisiensi, Proyeksi User-
User Unggul dalam Efektivitas Personalisasi.

======================================================================
SOAL 2.1.7 & 2.1.8: PERHITUNGAN SIMILARITY & QUERY OUTPUT NEO4J
======================================================================

--- Output 2.1.7 (a & b): 5 Record Similarity Teratas (Cypher Output) ---
 → User Acuan: 58 | User Mirip: 686 | Nilai Similarity: 1.0000
 → User Acuan: 572 | User Mirip: 934 | Nilai Similarity: 1.0000
 → User Acuan: 753 | User Mirip: 814 | Nilai Similarity: 1.0000
 → User Acuan: 466 | User Mirip: 949 | Nilai Similarity: 1.0000
 → User Acuan: 268 | User Mirip: 728 | Nilai Similarity: 0.7204

--- Output 2.1.8 Kondisi SEBELUM Perhitungan Similarity (Graf Bipartite
User-Item) ---
 → Visitor (257597) -[:VIEWED]→ Item (355908)
 → Visitor (992329) -[:VIEWED]→ Item (248676)
 → Visitor (111016) -[:VIEWED]→ Item (318965)
 → Visitor (483717) -[:VIEWED]→ Item (253185)
```

```
 → Visitor (951259) -[:VIEWED]→ Item (367447)

--- Output 2.1.8 Kondisi SESUDAH Perhitungan Similarity (Graf Terproyeksi
User-User) ---
 → User (58) -[:SIMILAR {score: 1.0000}]→ User (686)
 → User (572) -[:SIMILAR {score: 1.0000}]→ User (934)
 → User (753) -[:SIMILAR {score: 1.0000}]→ User (814)
 → User (466) -[:SIMILAR {score: 1.0000}]→ User (949)
 → User (268) -[:SIMILAR {score: 0.7204}]→ User (728)

═══ Bagian 2.1 Selesai Dijalankan ═══
```

In [6]:

```python
#
==========================================================================
==
# BAGIAN 2.2: ANALYTICS (REKOMENDASI KNN, Hit@5 & NDCG@5) - PERBAIKAN
#
==========================================================================
==

import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

print("=====================================================================")
print("SOAL 2.2.1: PEMILIHAN USER ACAK & KNN RECOMMENDATION (k=5)")
print("=====================================================================")

# 1. Cari kandidat user yang AKTIF di P1 DAN P2 (memiliki ground truth di
P2)
users_in_p1 = set(user_profiles['visitorid'])
users_in_p2 = set(df_p2['visitorid'].unique())
active_users_both = sorted(list(users_in_p1.intersection(users_in_p2)))

print(f"Total User Aktif di P1 & P2: {len(active_users_both)} pengunjung")

# 2. Proses pengacakan reproducible berbasis seed fixed
np.random.seed(2026)
# Pilih user acak dari daftar user yang aktif di kedua periode
target_visitor_id = int(np.random.choice(active_users_both))

# Dapatkan indeks baris target user di dataframe user_profiles P1
random_user_idx = user_profiles[user_profiles['visitorid'] ==
target_visitor_id].index[0]

print(f"1. Metode Pengacakan : Fixed Seed pada User Aktif P1 & P2
(np.random.seed(2026))")
print(f"2. User Terpilih     : Visitor ID {target_visitor_id}\n")

# 3. Hitung Cosine Similarity target user terhadap seluruh user P1
target_vsm = vsm_user_matrix[random_user_idx]
sim_scores = cosine_similarity(target_vsm, vsm_user_matrix).flatten()
```

```
# Mendapatkan k=5 Tetangga Terdekat
k = 5
knn_indices = np.argsort(sim_scores)[::-1][1:k+1]

print(f"--- Top-{k} Tetangga Terdekat (KNN) untuk Visitor
{target_visitor_id} ---")
neighbor_items = []
for rank, idx in enumerate(knn_indices, start=1):
    n_id = user_profiles.iloc[idx]['visitorid']
    score = sim_scores[idx]
    items = user_profiles.iloc[idx]['itemid'].split()
    neighbor_items.extend(items)
    print(f" Peringkat {rank}: Visitor ID {n_id} | Skor Similarity:
{score:.4f}")

# Item yang sudah diinteraksi Target User di Periode P1
target_p1_items = set(user_profiles.iloc[random_user_idx]['itemid'].split())

# 4. Filter 3 Item Rekomendasi/Iklan (Barang yang belum pernah diinteraksi
di P1)
recommended_items = []
for item in neighbor_items:
    if item not in target_p1_items and item not in recommended_items:
        recommended_items.append(item)
    if len(recommended_items) == 3:
        break

# Jika kandidat tetangga persis sama, ambil item paling populer dari
pengguna lain
if len(recommended_items) < 3:
    all_popular_items = [it for sublist in
user_profiles['itemid'].str.split() for it in sublist]
    for it in all_popular_items:
        if it not in target_p1_items and it not in recommended_items:
            recommended_items.append(it)
        if len(recommended_items) == 3:
            break

print(f"\n→ (b) 3 Item yang Diiklankan untuk Visitor {target_visitor_id}
(k=5 KNN): {recommended_items}\n")


# ------------------------------------------------------------------------
----
# SOAL 2.2.2: PERHITUNGAN METRIK Hit@5 DAN NDCG@5
# ------------------------------------------------------------------------
----
print("=================================================================")
print("SOAL 2.2.2: PERHITUNGAN METRIK Hit@5 DAN NDCG@5")
print("=================================================================")

# Ambil Ground Truth sebenarnya dari Target User di Periode P2 (Testing)
```

```
target_p2_ground_truth = set(df_p2[df_p2['visitorid'] == target_visitor_id]
['itemid'].astype(str).tolist())
print(f"1. Item Ground Truth Visitor {target_visitor_id} di Periode Testing
(P2): {list(target_p2_ground_truth)}")

# Lengkapi daftar rekomendasi hingga Top-5 item untuk evaluasi
top_5_recommendations = recommended_items.copy()
for item in neighbor_items:
    if item not in target_p1_items and item not in top_5_recommendations:
        top_5_recommendations.append(item)
    if len(top_5_recommendations) == 5:
        break

# Jika masih kurang dari 5, tambahkan item unik lainnya
if len(top_5_recommendations) < 5:
    for it in neighbor_items:
        if it not in top_5_recommendations:
            top_5_recommendations.append(it)
        if len(top_5_recommendations) == 5:
            break

print(f"2. Daftar Top-5 Rekomendasi Evaluasi: {top_5_recommendations}\n")

# (a) Perhitungan Hit@5
hits = [1 if item in target_p2_ground_truth else 0 for item in
top_5_recommendations]
hit_at_5 = 1.0 if sum(hits) > 0 else 0.0

# (b) Perhitungan NDCG@5
dcg = 0.0
for rank, is_hit in enumerate(hits, start=1):
    if is_hit:
        dcg += 1.0 / np.log2(rank + 1)

ideal_hits = min(len(target_p2_ground_truth), 5)
idcg = sum([1.0 / np.log2(rank + 1) for rank in range(1, ideal_hits + 1)])

ndcg_at_5 = (dcg / idcg) if idcg > 0 else 0.0

print("=== HASIL PERHITUNGAN METRIK EVALUASI REKOMENDASI ===")
print(f"(a) Hit@5  : {hit_at_5:.4f}")
print(f"(b) NDCG@5 : {ndcg_at_5:.4f}")
```

```
======================================================================
SOAL 2.2.1: PEMILIHAN USER ACAK & KNN RECOMMENDATION (k=5)
======================================================================
Total User Aktif di P1 & P2: 28747 pengunjung
1. Metode Pengacakan : Fixed Seed pada User Aktif P1 & P2
(np.random.seed(2026))
2. User Terpilih     : Visitor ID 113265

--- Top-5 Tetangga Terdekat (KNN) untuk Visitor 113265 ---
```

```
 Peringkat 1: Visitor ID 113265 | Skor Similarity: 1.0000
 Peringkat 2: Visitor ID 600755 | Skor Similarity: 1.0000
 Peringkat 3: Visitor ID 995624 | Skor Similarity: 1.0000
 Peringkat 4: Visitor ID 909839 | Skor Similarity: 1.0000
 Peringkat 5: Visitor ID 469324 | Skor Similarity: 0.0000

→ (b) 3 Item yang Diiklankan untuk Visitor 113265 (k=5 KNN): ['200425',
'164941', '139394']

======================================================================
SOAL 2.2.2: PERHITUNGAN METRIK Hit@5 DAN NDCG@5
======================================================================
1. Item Ground Truth Visitor 113265 di Periode Testing (P2): ['235154']
2. Daftar Top-5 Rekomendasi Evaluasi: ['200425', '164941', '139394',
'47006']

═══ HASIL PERHITUNGAN METRIK EVALUASI REKOMENDASI ═══
(a) Hit@5  : 0.0000
(b) NDCG@5 : 0.0000
```